Hybrid Uncertainty-Aware Network Diagnostic System

Mathematical Architecture — From Raw Dataset to Final Posterior

This document specifies the complete mathematical pipeline of the system, from synthetic data generation through to the final fused diagnosis. It is the formal specification behind `network_troubleshooting_datase` `network_troubleshooting_training.py`, `rule_based_engine.py`, `rf_uncertainty.py`, `bayesian_fusion.py`, and `hybrid_system.py`.

1 Pipeline Overview

Two experts, one fused posterior:

- **Branch A (statistical):** Random Forest over TF-IDF + structured features $\rightarrow P_{RF}(y \mid x)$
- **Branch B (symbolic):** RFC-grounded rule engine $\rightarrow P_{rule}(y \mid x)$
- **Uncertainty:** entropy of $P_{RF} \rightarrow u_{RF} \in [0, 1]$
- **Fusion:** consensus / override / Bayesian averaging $\rightarrow$ final $\hat{y}$, confidence, uncertainty

There are $C = 8$ diagnostic classes throughout:

$Y = \{\text{Router Issue}, \text{DNS Issue}, \text{DNS Timeout}, \text{IP Conflict}, \text{DHCP Failure}, \text{Gateway Unreachable}, \text{Network Adapte}$

2 Dataset Generation — `network_troubleshooting_dataset.py`

The dataset is a **class-conditional generative model**: for each class $y \in Y$ a distinct generating distribution $p(x \mid y)$ is defined, and samples are drawn class-by-class in equal quotas, then shuffled.

2.1 Feature decomposition

$$x = (t,\, b,\, c,\, z)$$

where t is the free-text symptom description, $b \in \{0, 1\}^7$ are binary diagnostic flags, $c \in \mathbb{R}^3$ are continuous diagnostic metrics, and z are two categorical context variables.

2.2 Text component

For class y, a keyword bank $K_y = \{k_1, \ldots, k_{20}\}$ is defined, and

$$t \mid y \sim \text{Uniform}(K_y)$$

2.3 Binary flags — fixed fingerprint + noisy channel

Each class has a fingerprint $b_y^* \in \{0,1\}^7$. Every bit passes independently through a binary symmetric channel with crossover probability $\varepsilon = 0.02$:

$$b_i = \begin{cases} b_i^* & \text{with probability } 1-\varepsilon \\ 1-b_i^* & \text{with probability } \varepsilon \end{cases}$$

2.4 Continuous metrics — class-conditional uniform intervals

$$c_k \mid y \sim \text{Uniform}[\, lo_{y,k},\ hi_{y,k}\,], \qquad k \in \{\text{dns_response_time_ms},\ \text{packet_loss_pct},\ \text{traceroute_hops}\}$$

When two classes share an identical binary fingerprint ($b_{y_i}^* = b_{y_j}^*$), separability is possible **only** through non-overlapping interval support:

$$[lo_{y_i,k}, hi_{y_i,k}] \cap [lo_{y_j,k}, hi_{y_j,k}] \approx \varnothing$$

2.5 Categorical context (pure confounders)

$$z_1 \sim \text{Uniform}(\{\text{WiFi}, \text{Ethernet}\}), \qquad z_2 \sim \text{Uniform}(\{\text{Windows}, \text{macOS}, \text{Linux}\})$$

$$I(z_1; y) = I(z_2; y) = 0$$

2.6 Class balance and empirical prior

$$N_y = \left\lfloor \frac{N}{8} \right\rfloor \ \ \forall y \qquad \Longrightarrow \qquad \hat{P}(y) = \frac{N_y}{N} \approx \frac{1}{8} \ \ \forall y$$

This uniform empirical prior is reused later as the Bayesian prior $P(y)$ in fusion (Section 6).

3 Feature Engineering — `network_troubleshooting_training.py`

3.1 Categorical label encoding

$$\text{LabelEncode} : \Sigma_j \to \{0, 1, \ldots, |\Sigma_j| - 1\}$$

3.2 TF–IDF vectorization

For vocabulary size $V_{max} = 100$ (unigrams + bigrams, min_df = 2):

$$\text{TF-IDF}(w,t) = tf(w,t) \cdot idf(w)$$

$$tf(w,t) = \text{count of } w \text{ in } t$$

$$idf(w) = \ln\left(\frac{1+n}{1+df(w)}\right) + 1$$

where n is the number of training documents and $df(w)$ the number of documents containing w. Each document vector is L2-normalized:

$$v(t) = [\text{TF-IDF}(w_1,t), \ldots, \text{TF-IDF}(w_V,t)] \in \mathbb{R}^V, \qquad \hat{v}(t) = \frac{v(t)}{\|v(t)\|_2}$$

3.3 Feature concatenation

$$\Phi(x) = \hat{v}(t) \ \oplus \ b \ \oplus \ c \ \oplus \ \hat{z} \ \in \ \mathbb{R}^{V+7+3+2} = \mathbb{R}^{112}$$

3.4 Stratified split

$$X_{train}, X_{test}, y_{train}, y_{test} = \text{split}(X, y;\ \text{test_size} = 0.2,\ \text{stratify} = y)$$

$$\frac{|\{i \in test : y_i = y\}|}{|test|} \approx \frac{|\{i \in train : y_i = y\}|}{|train|} \approx \frac{1}{8} \qquad \forall y$$

4 Base Statistical Learner — Random Forest

4.1 Gini impurity splitting

For a node with sample set S and class proportions p_k:

$$G(S) = 1 - \sum_{k=1}^{C} p_k^2$$

For a candidate split into S_L, S_R:

$$\Delta G(\text{split}) = G(S) - \left(\frac{|S_L|}{|S|} G(S_L) + \frac{|S_R|}{|S|} G(S_R) \right)$$

Each node greedily picks the split maximizing ΔG, subject to min_samples_split $= 5$, min_samples_leaf $= 2$.

4.2 Bootstrap aggregation

$$D_j \sim \text{SampleWithReplacement}(D_{train}, |D_{train}|), \qquad j = 1, \ldots, T$$

$$\mathbb{E}\left[\frac{|\text{distinct examples in } D_j|}{|D_{train}|} \right] \approx 1 - \frac{1}{e} \approx 0.632$$

Balanced class weighting inside each tree:

$$w_k = \frac{N}{C \cdot n_k}$$

where n_k is class k's frequency within that tree's bootstrap sample.

4.3 Ensemble probability

$$P_{RF}(y \mid x) = \frac{1}{T} \sum_{j=1}^{T} p_j(y \mid x), \qquad T = 200$$

$$\hat{y}_{RF} = \underset{y \in Y}{\text{argmax}}\, P_{RF}(y \mid x), \qquad conf_{RF} = \max_y P_{RF}(y \mid x)$$

5 Uncertainty Quantification — `rf_uncertainty.py`

5.1 Predictive (total) entropy

$$H[P_{RF}(\cdot \mid x)] = -\sum_{y \in Y} P_{RF}(y \mid x) \cdot \ln P_{RF}(y \mid x)$$

(with an $\varepsilon = 10^{-10}$ floor per probability to avoid $\ln 0$).

5.2 Entropy normalization

Maximum entropy for $C = 8$ classes occurs at the uniform distribution:

$$H_{max} = -\sum_{y=1}^{C} \frac{1}{C} \ln \frac{1}{C} = \ln(C) = \ln(8) \approx 2.0794 \text{ nats}$$

Normalized uncertainty score:

$$u_{RF}(x) = \text{clip}\left(\frac{H[P_{RF}(\cdot \mid x)]}{\ln(C)}, 0, 1 \right)$$

Both numerator and denominator use natural log throughout — no bits/nats mismatch in this module.

5.3 Inter-tree variance

$$\text{Var}_j[p_j(y \mid x)] = \frac{1}{T} \sum_{j=1}^{T} \left(p_j(y \mid x) - P_{RF}(y \mid x) \right)^2$$

$$\sigma^2_{inter}(x) = \frac{1}{C} \sum_{y \in Y} \text{Var}_j[p_j(y \mid x)]$$

5.4 Mutual information (epistemic uncertainty) — BALD decomposition

Expected per-tree entropy:

$$\mathbb{E}_j[H[p_j(\cdot \mid x)]] = \frac{1}{T} \sum_{j=1}^{T} \left(-\sum_{y} p_j(y \mid x) \ln p_j(y \mid x) \right)$$

Mutual information:

$$I(x) = H[P_{RF}(\cdot \mid x)] - \mathbb{E}_j[H[p_j(\cdot \mid x)]]$$

$$MI(x) = \max(0,\, I(x))$$

Interpretation: $MI(x) \to 0$ means trees individually agree with the ensemble mean (pure aleatoric uncertainty). $MI(x)$ large means trees are each confident but disagree with each other (pure epistemic uncertainty).

5.5 Calibration diagnostics (reliability curve)

$$conf_i = \max_y P_{RF}(y \mid x_i), \qquad correct_i = \mathbb{1}\big[\operatorname*{argmax}_y P_{RF}(y \mid x_i) = y_i\big]$$

For $M = 10$ equal-width bins $B_m = \big[\frac{m}{M}, \frac{m+1}{M}\big)$:

$$\text{bin_confidence}(m) = \frac{1}{|B_m|} \sum_{i \in B_m} conf_i, \qquad \text{bin_accuracy}(m) = \frac{1}{|B_m|} \sum_{i \in B_m} correct_i$$

Expected Calibration Error:

$$ECE = \sum_m \frac{|B_m|}{N} \big|\text{bin_accuracy}(m) - \text{bin_confidence}(m)\big|$$

6 Symbolic Reasoning — `rule_based_engine.py`

6.1 Decision-list formalism

Ordered rules $r_1, \ldots, r_8$ with predicates $\pi_1, \ldots, \pi_8$ and fixed confidences $\kappa_1, \ldots, \kappa_8 \in (0, 1)$:

$$\text{diagnose}(x) = (y_m, \kappa_m, \text{reasons}_m), \qquad m = \min\{i : \pi_i(x) = 1\}$$

Example predicates:

$$\pi_{DHCP}(x) = \mathbb{1}[\text{has_ip} = 0] \wedge \mathbb{1}[\text{arp_table_ok} = 1]$$

$$\pi_{DNStimeout}(x) = \mathbb{1}[\text{ping_ip} = 1] \wedge \mathbb{1}[\text{ping_domain} = 0] \wedge \mathbb{1}[dns_rt(x) > 8000]$$

$$\pi_{DNSissue}(x) = \mathbb{1}[\text{ping_ip} = 1] \wedge \mathbb{1}[\text{ping_domain} = 0] \wedge \mathbb{1}[dns_rt(x) \leq 8000]$$

Priority	Class	κ
1	DHCP Failure	0.95
2	Network Adapter Issue	0.98
3	IP Conflict	0.90
4	Subnet Mismatch	0.88
5	Gateway Unreachable	0.85
6	DNS Timeout	0.82
7	DNS Issue	0.80
8	Router Issue	0.78

6.2 Full class-probability vector

Step 1 — winning class gets the fired confidence:

$$P_{rule}(y_m \mid x) = \kappa_m$$

Step 2 — partial score for each non-winning class (example, Subnet Mismatch):

$$s(\text{SubnetMismatch}, x) = 0.6 \cdot \mathbb{1}[\text{subnet_ok} = 0] + 0.4 \cdot \mathbb{1}[\text{ping_gw} = 0 \wedge \text{has_ip} = 1]$$

Weighted proportional split of remaining mass:

$$P_{rule}(y \mid x) = \begin{cases} \dfrac{(1-\kappa_m)\cdot s(y,x)}{\sum_{y' \neq y_m} s(y',x)} & \text{if } \sum_{y'} s(y',x) > 0 \\ \dfrac{1-\kappa_m}{C-1} & \text{if } \sum_{y'} s(y',x) = 0 \quad \text{(uniform fallback)} \end{cases}$$

Final renormalization (numerical safeguard):

$$P_{rule}(y \mid x) \leftarrow \frac{P_{rule}(y \mid x)}{\sum_{y' \in Y} P_{rule}(y' \mid x)}$$

6.3 Heuristic fallback

If no π_i fires, a two-branch heuristic on packet_loss_pct and ping_gateway assigns a diagnosis with reduced confidence (0.50–0.55), signalling incomplete rule coverage to the fusion stage.

7 Bayesian Model Fusion — `bayesian_fusion.py`

Inputs: $P_{RF}(y \mid x)$ with uncertainty $u_{RF} \in [0,1]$, and $P_{rule}(y \mid x)$ with uncertainty proxy $u_{rule} = 1 - \max_y P_{rule}(y \mid x)$. Threshold $\tau = 0.30$.

7.1 Case 1 — Consensus ($\hat{y}_{RF} = \hat{y}_{rule} = y^*$)

Certainties:

$$c_{RF} = 1 - u_{RF}, \qquad c_{rule} = 1 - u_{rule}$$

Normalized weights:

$$w_{RF} = \frac{c_{RF}}{c_{RF} + c_{rule}}, \qquad w_{rule} = \frac{c_{rule}}{c_{RF} + c_{rule}}$$

Combined confidence with agreement boost $\beta = 1.05$:

$$conf(y^*) = \min\Big(1,\ \beta\big(w_{RF}\cdot P_{RF}(y^* \mid x) + w_{rule}\cdot P_{rule}(y^* \mid x)\big)\Big)$$

7.2 Case 2 — Uncertainty-gated override ($\hat{y}_{RF} \neq \hat{y}_{rule}$)

$$\text{confident}_{RF} = \mathbb{1}[u_{RF} < \tau], \qquad \text{confident}_{rule} = \mathbb{1}[u_{rule} < \tau]$$

$$(\hat{y}, conf) = \begin{cases} (\hat{y}_{RF},\ P_{RF}(\hat{y}_{RF} \mid x)) & \text{if confident}_{RF} = 1,\ \text{confident}_{rule} = 0 \\ (\hat{y}_{rule},\ P_{rule}(\hat{y}_{rule} \mid x)) & \text{if confident}_{rule} = 1,\ \text{confident}_{RF} = 0 \end{cases}$$

7.3 Case 3 — Full Bayesian Model Averaging

Triggered when both experts are confident-but-disagreeing, or both uncertain. Global reliabilities: $\text{Rel}_{RF} = 0.996$, $\text{Rel}_{rule} = 0.880$.

Instance-level weights:

$$\tilde{w}_{RF} = (1-u_{RF})\cdot \text{Rel}_{RF}, \qquad \tilde{w}_{rule} = (1-u_{rule})\cdot \text{Rel}_{rule}$$

$$w_{RF} = \frac{\tilde{w}_{RF}}{\tilde{w}_{RF} + \tilde{w}_{rule}}, \qquad w_{rule} = \frac{\tilde{w}_{rule}}{\tilde{w}_{RF} + \tilde{w}_{rule}}$$

(fallback $w_{RF} = w_{rule} = 0.5$ if $\tilde{w}_{RF} + \tilde{w}_{rule} = 0$)

Posterior (Bayes' rule with a surrogate likelihood):

$$\text{likelihood}(y \mid x) = w_{RF} \cdot P_{RF}(y \mid x) + w_{rule} \cdot P_{rule}(y \mid x)$$

$$\text{posterior}_{unnorm}(y \mid x) = P(y) \cdot \text{likelihood}(y \mid x)$$

Normalization:

$$P_{fused}(y \mid x) = \frac{\text{posterior}_{unnorm}(y \mid x)}{\sum_{y' \in Y} \text{posterior}_{unnorm}(y' \mid x)}$$

Final decision:

$$\hat{y} = \underset{y \in Y}{\text{argmax}}\, P_{fused}(y \mid x), \qquad conf = P_{fused}(\hat{y} \mid x)$$

7.4 Summary of the three regimes

Regime	Trigger	Result
Consensus	$\hat{y}_{RF} = \hat{y}_{rule}$	certainty-weighted average $\times\ \beta = 1.05$
Override	disagree, exactly one confident ($u < \tau = 0.30$)	that expert's raw $(\hat{y}, conf)$
Bayesian fusion	disagree, both/neither confident	full BMA posterior $P_{fused}(y \mid x)$

8 End-to-End Trace — `hybrid_system.py`

1. **Feature construction:** $x = (t, b, c, z) \to \Phi(x) \in \mathbb{R}^{112}$ for the RF arm; raw feature dict for the rule engine.
2. **Rule arm:** $(\hat{y}_{rule}, conf_{rule}, \text{reasons}) = \text{diagnose_rules}(x)$, and $P_{rule}(\cdot \mid x)$.
3. **RF arm:** $P_{RF}(\cdot \mid x) = \frac{1}{T}\sum_j p_j(\cdot \mid x)$, $\hat{y}_{RF} = \text{argmax}_y\, P_{RF}(y \mid x)$.
4. **Uncertainty:** $H[P_{RF}] \to u_{RF} = H/\ln(8)$, plus $\sigma^2_{inter}(x)$ and $MI(x)$ surfaced for explainability (not consumed by the fusion decision itself, which only uses the scalar u_{RF}).
5. **Fusion:** $(\hat{y}, conf, \text{strategy}) = \text{combine}(P_{RF}, P_{rule}, u_{RF}, u_{rule})$, with $u_{rule} = 1 - conf_{rule}$ when not supplied.
6. **Fallback:** if the RF pipeline is unavailable or fails at inference, $\hat{y} = \hat{y}_{rule}$, $conf = conf_{rule}$, $u = 1 - conf_{rule}$ — mathematically equivalent to forcing $w_{RF} = 0, w_{rule} = 1$ in Case 3.
7. **Output:** $\hat{y}$, $conf$, u_{RF}, fusion strategy label, full distribution $P_{fused}(\cdot \mid x)$ (or P_{RF}/P_{rule} in fallback paths), plus the rule trace and entropy/MI/variance breakdown.

9 Consolidated Notation Reference

Symbol	Meaning
$C = 8$	number of diagnostic classes
$T = 200$	number of trees in the Random Forest
$V = 100$	TF-IDF vocabulary size
$\Phi(x) \in \mathbb{R}^{112}$	full RF feature vector
$P_{RF}(y \mid x)$	Random Forest class-probability vector
$P_{rule}(y \mid x)$	rule engine's class-probability vector
$H[p] = -\sum p \log p$	Shannon entropy (natural log)
$\ln(C) = \ln 8 \approx 2.079$	maximum entropy for 8 classes
$u_{RF} = H[P_{RF}]/\ln(C)$	normalized RF uncertainty $\in [0, 1]$
$MI(x)$	epistemic uncertainty (BALD mutual information)
$\sigma^2_{inter}(x)$	mean inter-tree class-probability variance
$\tau = 0.30$	confidence/uncertainty threshold for override regime
$\beta = 1.05$	agreement boost multiplier (consensus regime)
w_{RF}, w_{rule}	instance-level fusion weights
$\text{Rel}_{RF} = 0.996,\ \text{Rel}_{rule} = 0.880$	held-out global reliability priors
$P_{fused}(y \mid x)$	final posterior distribution after fusion